

The Optimizations Behind cMVBT-OSIC: A Unified, Verified Account

Amir Tonta

12 August 2026

Abstract

This document consolidates every optimization-focused write-up in this repository — the narrative design document (`docs/optimizations.tex`), the tabular inventory (`docs/optimization_catalog.tex`), the compact implementation guide (`docs/index_optimizations.md`), and five focused microbenchmark reports — into one single account of the entire cMVBT-OSIC implementation. It is organized neither by commit nor by document of origin, but bottom-up by architectural layer, exactly the order a reader would need to actually understand the system: physical page layout, then concurrency control built on top of it, then structural maintenance, then the transaction layer, then garbage collection, then the cross-cutting sharding and durability stories that sit above all of it. Every optimization is explained (what changed, why, and what it cost or risked), not merely listed. A dedicated section (§2) argues explicitly for four optimizations that are *foundational*: removing any one of them would not just cost some throughput, it would make several other optimizations described later in this document either meaningless or unsound. Wherever the repository’s own history contains a controlled before/after measurement, that number is reported with its workload, thread count, and caveats attached. Wherever it was practical to do so within this session, those numbers were independently re-measured on the machine this document was written on (§16); the two do not always agree, and both are reported, since the disagreement itself is informative. This document also surfaces optimization attempts that were tried and reverted, and issues that remain open and unfixed as of this writing — an accurate account of the whole implementation has to include the parts that did not work as well as the parts that did.

Contents

1	Introduction and how to read this document	3
2	Foundational optimizations: what everything else stands on	4
2.1	Append-only, publish-once physical pages	4
2.2	Optimistic lock coupling and the SmartCell latch	4
2.3	OSIC: decoupling visibility from durability	5
2.4	A per-worker identity substrate	5
2.5	A second-order case: self-overwrite	5
3	Physical layer: pages and multiversion records	6
3.1	Append-only leaves and the version tuple	6
3.2	Structure-of-arrays leaf layout	6
3.3	Inline validity mask	6
3.4	Inline internal-page liveness mask	6
3.5	Exact 4 KiB allocation, payload indirection, and the hot/cold split	7
3.6	Measured: leaf layout, isolated	7

4	Concurrency control: from lock coupling to optimistic lock coupling	8
4.1	The protocols that were tried	8
4.2	The rise and fall of ORWC — a cautionary tale	8
4.3	The SmartCell latch	8
4.4	Chasing fences out of the hot path	8
5	Structural maintenance: splits, merges, and page capacity	9
5.1	Capacity tuning	9
5.2	Split: choosing where to cut	10
5.3	Merge: choosing what to combine	10
5.4	A week of hardening (2026-07-31 – 08-02)	10
6	The self-overwrite optimization	10
6.1	The problem	11
6.2	The fix, as it exists in the current code	11
6.3	Delete/reinsert cycles use the same bound	12
6.4	The chronological story	12
6.5	Resulting bound, and a known residual	12
7	Garbage collection: reclaiming dead blocks without contention	13
7.1	From a global mutex to sharded, lock-free reclaim	13
7.2	Two ThreadSanitizer races, closed structurally	13
7.3	Sharded reclaim, historical routing, and abort-safe retention	13
8	Root indexing strategies	14
9	Sharding as a system-wide pattern	14
9.1	Tables as shards: one tree per table	14
9.2	GC dead-block tracker: own-shard-first, round-robin steal	14
9.3	TxContext: cache-padded, per-worker atomic slots	15
9.4	Thread-local worker identity and snapshot cache	15
10	OSIC integration: instant commit, lazily hardened	15
10.1	What OSIC is, and how cMVBT realizes it	15
10.2	The WAL performance campaign	16
10.3	A lock-free WAL backend, and two concurrency bugs it took to land	17
10.4	Ticket-less logging and exact framing hints	17
10.5	Two dead ends, honestly reported	18
10.6	Atomic operations: an auto-commit fast path alongside real transactions	18
10.7	Known, documented limitations	19
11	Range scans and analytical terminals	19
11.1	Why internal pages cannot be scanned in physical order	19
11.2	Rejected: child-list reuse, small-page dedup, hybrid traversal, lean frames	19
11.3	Streaming terminals: zero-copy analytical consumption	19
11.4	Fix 1: test the range before the visibility check	20
11.5	Fix 2: remove the dyn FnMut indirection	20
11.6	The point-read miss bug this work incidentally surfaced	20
12	Workload and benchmark-harness optimizations	21
13	Case study: BigTreeSize — a contention/garbage continuum, worked end to end	22

13.1 Motivation	22
13.2 The current design: an eight-variant size-class enum	22
13.3 A benchmark methodology bug, found and corrected mid-investigation	24
13.4 Recommendation	24
14 Rejected, reverted, or deliberately optional experiments	24
15 Open issues, as of this writing	26
16 Fresh measurements, this session	27
16.1 Mechanism presence check	27
16.2 Full test suite	27
16.3 Leaf layout benchmark, independently reproduced	27
16.4 WAL writer throughput, independently reproduced	28
17 Combined measured gains at a glance	29
18 Timeline	30
19 Conclusion	31

1 Introduction and how to read this document

cMVBT is a concurrent, in-memory, multiversion B-tree implementing the asymptotically optimal MVBT structure of Becker et al. [1], extended with the multiversion concurrency control protocol of [2]. On top of the tree, the system integrates *Ordered Snapshot Instant Commit* (OSIC), the visibility/durability model from LeanStore’s scalable snapshot-isolation work [3]: commits become visible instantly, in commit order, while the write-ahead log is hardened to disk lazily and in batches, off the commit’s critical path.

Roadmap. Sections build on one another in this order:

1. **Foundational optimizations** (§2): four decisions that everything downstream either depends on for correctness or depends on for the optimization to matter at all.
2. **Physical layer** (§3): how one page holds several versions of a key, and how that page is laid out in memory.
3. **Concurrency control** (§4): how threads touch pages from §3 without unnecessary blocking.
4. **Structural maintenance** (§5): splitting and merging pages as they fill up or empty out.
5. **The self-overwrite optimization** (§6): bounding how much physical growth one transaction’s repeated writes can cause — the second load-bearing optimization this document singles out, one layer up from §2.
6. **Garbage collection** (§7): reclaiming pages retired by §5.
7. **Root indexing** (§8) and **sharding** (§9): finding the right root, and the recurring pattern of partitioning shared bookkeeping state by worker.
8. **OSIC and the WAL** (§10): the commit/durability layer, including a full accounting of the WAL performance work.
9. **Range scans** (§11) and **workload/benchmark-harness optimizations** (§12).

10. **Case study: BigTreeSize** (§13): one optimization followed end to end, including a benchmark methodology bug that was found and fixed mid-investigation.
11. **Rejected experiments** (§14) and **open issues** (§15): what was tried and did not work, and what is still broken.
12. **Fresh measurements** (§16), a **combined gains table** (§17), a **timeline** (§18), and the **conclusion** (§19).

Evidence levels. As in the earlier catalog, every claimed gain is tagged: **Measured** (a before/after benchmark exists, with its workload and caveats), **Bounded** (a countable cost — an allocation, a lock, a retry, a scan — is removably bounded, but no trustworthy end-to-end percentage was recorded), or **Enabling** (primarily a correctness or livelock/deadlock fix, not a claimed speedup). Combined benchmark results are never attributed to one component in isolation unless the repository contains a measurement that isolates it.

2 Foundational optimizations: what everything else stands on

Reading the rest of this document commit-by-commit gives the impression of dozens of independent, similarly-weighted improvements. They are not independent. Four design decisions sit underneath nearly everything else described later, in the specific sense that removing any one of them would not merely cost throughput — it would make a named optimization described in a later section either *unsound* (its correctness argument stops holding) or *pointless* (the cost it removes stops existing in the first place). This section names all four before the rest of the document goes layer by layer through the system; each subsection below points forward to exactly which later optimizations it underwrites.

2.1 Append-only, publish-once physical pages

Every leaf page (`src/mv_page_model/leaf_page.rs`) is a fixed-capacity array that is only ever appended to. An obsolete, aborted, or deleted entry is *marked* dead via `VersionInfo::delete/invalidate`; it is never physically overwritten or removed until the next structural-maintenance pass rebuilds the page from scratch (§3). Internal pages follow the same rule: a superseded child entry is never mutated, only shadowed by a newer appended entry (§7).

What this underwrites. Optimistic lock coupling (§4) lets readers inspect a page with no latch at all specifically because nothing they might observe was ever mutated after being published — a reader racing a writer either sees the old, still-fully-formed state or the new one, never a half-written slot. Garbage collection’s internal-page liveness derivation (`live_mask()`, §7) is a pure function of *disjoint, append-ordered intervals* — it has no meaning if slots could be mutated in place. And the self-overwrite optimization’s entire safety argument (§6) is the claim “nobody else could possibly be relying on the current content of this uncommitted tuple”; that claim presupposes a notion of “this transaction’s own, exclusively owned tuple” that only exists because writes append rather than clobber. Take this foundation away and every one of those three later sections would need a completely different, almost certainly lock-based, design.

2.2 Optimistic lock coupling and the SmartCell latch

Readers take an `Acquire` version sample of a node’s `SmartCell` (`src/mv_sync/smart_cell.rs`) and validate it after use, with no latch acquired at all; writers upgrade to an exclusive, seqlock-style write lock only on the node they actually mutate (§4). This one protocol survives out of six that were prototyped (§4).

What this underwrites. Every sharding optimization in §9 — per-worker `TxContext` slots, GC’s per-shard dead-block index, cache-padding those slots against false sharing — is a fix for contention *among concurrent, lock-free readers and writers*. If reads instead took a page latch (as in the discarded `LockCoupling/LHL/HL` protocols), most of that contention would already be serialized by the latch, and sharding the bookkeeping around it would buy nothing. The entire fence-removal chronology (Table 1) is similarly parasitic on OLC existing at all: those fixes remove ordering that OLC’s own atomics already subsumed, which is only a real saving because OLC’s hot path runs on every read.

2.3 OSIC: decoupling visibility from durability

Every transaction draws a start and a commit timestamp from one global atomic counter; a write becomes visible to other workers the instant its commit timestamp lands in the committing worker’s append-only `CommitLog` — an in-memory, $O(1)$ step with no wait on disk I/O (§10).

What this underwrites. The self-overwrite fast path’s detection rule, `insertion_stamp()=stamp`, is a comparison of two `TxStamp` values — a concept (`(worker\_id,ts\_start)`, constant for a whole transaction) that *is* an OSIC concept; without OSIC’s stamp model there is no single equality check that means “did I, this very transaction, write this” at all. More broadly, the entire WAL performance campaign in §10 — sharding the write channel, removing a discarded flush ticket, right-sizing framing buffers, an alternative lock-free backend — is optimizing the cost of *catching up durability asynchronously*. If commit instead required a synchronous `fsync` round-trip (the naive MVCC alternative OSIC replaces), every one of those WAL micro-optimizations would be dwarfed by that single round-trip’s latency by one to two orders of magnitude, and none of them would have been worth attempting first.

2.4 A per-worker identity substrate

A `WorkerId` is assigned once per (tree, thread) and cached thread-locally (`src/mv_sync/worker.rs`); it is the index into every per-worker structure the rest of the system builds (§9).

What this underwrites. Sharding, as a pattern, needs something to shard *along*. GC’s dead-block tracker shards by `worker_id\%shards.len()`; `TxContext`’s live-snapshot and in-flight-registration slots are one array entry per `WorkerId`; the thread-local `SnapshotCache` is keyed the same way. None of those three later optimizations is a response to “things are shared and contended” in the abstract — they are specifically the removal of contention on structures that used to be indexed by nothing (a single global mutex or counter) and are now indexed by this one identity. Without a stable per-worker identity assigned once and reused, each of those shards would need its own separate discovery mechanism, and the uniform “own-shard-first, steal on miss” idiom used throughout §9 would not generalize across them the way it currently does.

2.5 A second-order case: self-overwrite

One further optimization is worth flagging here even though it is not, by the standard above, foundational in the same primary sense — it depends on the four foundations above, rather than being depended on by everything. It is nonetheless *load-bearing for a specific later section*: without the self-overwrite fast path (§6) bounding one transaction’s repeated writes to a single key at one extra physical tuple, the structural-maintenance tuning in §5 — careful capacity thresholds, tiered boundary search for splits, emptier-sibling selection for merges — would be fighting an unbounded adversary: a single transaction that updates one hot key enough times inside itself can, without this fix, leave a page with no valid split boundary at all (§6 explains the exact failure shape). All of §5 assumes it is dealing with *bounded* per-key physical chains; self-overwrite is what keeps that assumption true.

3 Physical layer: pages and multiversion records

3.1 Append-only leaves and the version tuple

A leaf’s record array (`leaf_page.rs`) is append-only, as established in §2; a packed `len` counter tracks active vs. dead slots. Each tuple carries a `VersionInfo`: an `insertion_stamp:TxStamp` (a packed `(worker_id,ts_start)`, constant for the whole transaction — not per write, which is exactly what makes the self-overwrite check in §6 a single equality test) and an optional `deletion_stamp`. Lookups scan *backward* (`rfind`) for the newest live entry, since physical append order places the newest write last — this backward-scan invariant is relied on by merge (§5, which must not re-sort records by `ts_start`) and by the point-read miss fix discussed in §11.

Structural maintenance decides what a rebuild keeps via `record_survives_gc`: a record survives if it is live, *or* its deleting transaction is still unresolved (may still abort, and abort needs the predecessor to undelete). Readers are not the reason old records are copied forward during a rebuild — they instead route through the retired source block until they finish, at which point that block becomes reusable (§7).

3.2 Structure-of-arrays leaf layout

A leaf’s physical layout is a dense key region followed by a parallel version/payload region, not one array of complete `\{key,version,payload\}` records:

<code>key_region:</code>	<code>[key0, key1, key2, ...]</code>
<code>data_region:</code>	<code>[(version0, payload0), (version1, payload1), ...]</code>

Point lookup walks the key region backward; version metadata and payload cache lines are touched only *after* a key matches, and a range scan can test its key range before loading the parallel data at all. For `u64` keys and a one-word payload slot, one physical entry is 32 bytes (8 bytes key, 24 bytes version+payload) — the same per-slot footprint as the earlier array-of-structures layout, so this is a pure locality change, not a capacity change.

3.3 Inline validity mask

A normal leaf has at most 123 physical slots, so two `AtomicU64s` (128 bits) cover every slot’s *physical* validity (cleared on abort) directly inside the page, replacing an earlier boxed-slice handle and its pointer chase. This is explicitly *not* an MVCC visibility mask — visibility also depends on the reading transaction’s snapshot, which one universal per-slot bit cannot represent — it only lets lookup skip an aborted slot before loading its version header at all. Deliberately oversized TPC-C leaf variants (§13) fall back to a heap bitmap through a same-sized union so the normal 4KiB leaf does not pay for it.

3.4 Inline internal-page liveness mask

Internal pages are append-only across structural versions: replacing a child appends one or more new intervals while the old interval stays physically present for older snapshots. An earlier design tracked which entries were *currently* live with a separately mutated, in-place “obsolete mark” atomic bit on an already-published, concurrently-read slot — confirmed by ThreadSanitizer as a genuine race against range-query iteration (§7, `fe03d90`). The current design (`InternalPage::live_mask()`) instead derives liveness from the invariant that *live sibling key intervals are always pairwise disjoint*: for every appended interval, clear the bit of every older interval it overlaps, set its own bit, and publish with the existing Release length store. This is an $O(\text{FAN_OUT}^2)$ pure-integer pass that runs only on the SMO cold path (append), so every later SMO consumer

— merge-candidate selection, survivor counting, split copying — reads `is_slot_live(i)` as one word load and bit test instead of rebuilding a temporary `Vec<bool>` from scratch each time. Historical routing deliberately does *not* consult this mask: a child superseded in the *current* partition may still be the correct route for an older snapshot, so historical reads keep selecting via immutable per-entry versions (§11).

3.5 Exact 4 KiB allocation, payload indirection, and the hot/cold split

The 4 KiB target applies to the full allocated `OptCell<Block<...>>`, including the OLC `cell_version` word and node metadata — not just the record array. With `FAN_OUT=NUM_RECORDS=123`, the cell lands at exactly 4096 bytes; 124 records would cross that boundary. This single two-record trim (`b29d347`, `125` → `123`) is **Measured**: page-aligned allocations rose from 25% to 100% of the time, with a 9–12% reduction in dTLB load misses — concrete evidence that “nominally full” and “lands cleanly on a hardware page” are two different numbers, and only the second one is free.

A one-word `PayloadSlot` stores a `u64` or the thin YCSB row handle inline; larger TPC-C rows go through a reference-counted (`trionphe::Arc`) pointer instead, so cloning a read result or copying a survivor during a split increments a `refcount` rather than copying a whole row. `MVBTst` itself is split into a hot part (root, allocator, transaction context — everything the traversal-critical path touches) and a `cold:Box<_>` part (WAL handle, table id, construction-time bounds) purely to shrink the cache footprint of the hot path.

3.6 Measured: leaf layout, isolated

The split (structure-of-arrays) layout’s benefit was isolated in a standalone, dependency-free microbenchmark (`tools/leaf_layout_bench.rs`) comparing it against the earlier array-of-structures layout, with no tree traversal, latching, WAL, GC, transactions, allocation, or payload cloning in the timed region. 16,384 independently boxed leaf pages per layout (a 62.5 MiB working set each), five update waves per key (append-ordered, matching real leaves), point lookups walking backward, scans checking the range before visibility.

Original result (AMD Ryzen 9 5900X, median of 5 runs, 750 ms/operation):

Operation	AoS		Split	Speedup
Point: latest visible	30.57 Mlookups/s	33.17 Mlookups/s		1.085×
Point: 1-version fallback	18.43 Mlookups/s	20.15 Mlookups/s		1.093×
Point: 3-version fallback	9.03 Mlookups/s	11.02 Mlookups/s		1.221×
Scan: full leaves	322.84 Mslots/s	320.65 Mslots/s		0.993×
Scan: 20% key ranges	354.82 Mslots/s	389.09 Mslots/s		1.097×

The interpretation offered at the time: point lookups improve because the key region can reject non-matching entries without touching the version/payload cache lines at all, and the effect grows with how many dead versions a lookup has to walk past (8.5%, 9.3%, 22.1% for 0/1/3 fallbacks); full scans are unaffected because they touch every byte of the leaf regardless of layout.

Independent reproduction, this session (§16 has the full protocol and discussion): on a different, older CPU (AMD Ryzen 7 2700X, Zen+ rather than Zen 3), five repeated full-binary runs of the identical benchmark were extremely internally consistent (well under 1% run-to-run variance) but told a *smaller and partly reversed* story: the 62.5 MiB multi-leaf point-lookup gains replicated in direction but at roughly a third of the claimed magnitude ($1.03\times$ – $1.08\times$, not $1.09\times$ – $1.22\times$), and two of three scan variants came out *slower* for the split layout rather than faster ($0.93\times$ – $0.97\times$). This is very likely a real hardware effect, not a methodology error — see §16 for the reasoning — and it is reported here precisely because a document claiming to cover

the whole implementation should not silently drop a result that failed to reproduce as cleanly as hoped.

4 Concurrency control: from lock coupling to optimistic lock coupling

4.1 The protocols that were tried

A pre-multiversion sister crate, `CCBPlusTree`, defines six lock-mode variants: `MonoWriter` (no real concurrency control), `LockCoupling` (classic hand-over-hand exclusive latching), `ORWC` (“Optimistic Read-Write Coupling”), `OLC` (Leis-style optimistic lock coupling), and two level-bounded hybrids, `LHL/HL`. Only **OLC** survives in the current codebase’s write/read fast paths (`traversal_write_olc`, `traverse_read_key`, `try_retire`, `upgrade_write_lock`). `MonoWriter/LockCoupling/LHL/HL` were prototyped in early 2024 but never carried into the multiversion rewrite.

4.2 The rise and fall of ORWC — a cautionary tale

ORWC is worth its own subsection because it was implemented, iterated on for roughly a year, and then deliberately removed. `d565b2d` (2024-01-22) is the first sign of trouble (“*ORWC seem to have an issue*”); over the following sixteen months, four separate fix attempts addressed a root-upgrade validation gap, stripped redundant fences once OLC’s own atomics were trusted, reworked guards from raw pointers to cloned `SmartCell` handles after a dangling-pointer segfault on free readers, and finally gave up on non-blocking semantics for the ordinary mutable-borrow path, keeping only the optimistic *upgrade* non-blocking. `3c6da7b` (2024-12-06) deprecates it; `5c26abb` (2024-12-08) purges it outright, ~460 lines removed, with the commit message stating the reason plainly: “*duplicate with OLC.*” **Root cause, in one sentence:** ORWC’s optimistic upgrade path never reached a provably correct state, and once OLC was fully validated it gave the identical guarantee through one simpler seqlock mechanism — ORWC became pure maintenance liability, not a genuinely distinct protocol.

4.3 The SmartCell latch

`src/mv_sync/smart_cell.rs` implements the versioned latch every node uses: a single 64-bit `cell_version` word (`OptCell<E>\{cell, cell_version\}`) with three logical bit ranges packed into its top nibble — a seqlock-style write flag, pin-related flags, and a *permanent* retired-tombstone bit. An optimistic read (`borrow_read`) is one `Acquire` load with the write/retired bits masked out, no lock taken. Upgrading to exclusive (`upgrade_write_lock`) is a CAS from the captured read version to the write-flagged version, failing fast if the retired bit is already set. Unlock stores (`write\_version+1`)\^\`WRITE\_FLAG|retired\_bit` with `Release`, deliberately reading the current retired bit with a plain `Relaxed` load (safe, since nothing else can touch `cell_version` while the writer guard is held) so a same-critical-section `mark_retired()` call is preserved rather than clobbered by the unlock store (§7 covers the bug this specifically fixes). `try_retire` is a single CAS straight from a reader snapshot to the retired state, skipping the intermediate exclusive step — sound only for guards that are unconditionally consumed with no back-out path.

4.4 Chasing fences out of the hot path

Once OLC’s own atomics were trusted to carry the needed ordering, standalone memory fences bolted on earlier, out of caution, were incrementally identified as redundant and removed:

Commit	Date	Change	Rationale
48c4ded	2024-05-06	Removed <code>fence(Acquire)/fence(SeqCst)</code> in the traversal loop	The surrounding OLC load/CAS already provides the ordering.
3c6da7b	2024-12-06	Removed a paired fence duplicating <code>len.load(Acquire)</code>	An atomic load's own ordering already suffices.
a7bb29c	2025-04-01	CAS failure ordering <code>Relaxed→Acquire</code> ; unlock store <code>Relaxed→Release</code>	Establishes the exact happens-before edges OLC readers rely on, cutting spurious CAS retries.
a518eed	2025-04-25	Deleted standalone fences immediately adjacent to an atomic op on the same word	Pure dead weight once the adjacent op's ordering subsumed it.
2a2be93	2026-06-22	Soft-commit length bumps <code>Release→Relaxed</code>	The following write-unlock's own <code>Release</code> already publishes it.
40b00fd	2026-06-30	<code>Node::m_type AtomicUsize→usize</code> ; removed the Drop fence	Only mutated under the exclusive write lock; the latch's own release already publishes it.
45c7c21/fd922086	2026-07-21	<code>try_retire</code> lets unconditionally-consumed guards skip a full write-lock upgrade during split/merge	Removes one CAS/lock-acquire per split/merge for guards with no backout path.

Table 1: Chronology of fence removal and ordering tightening.

Alongside this, 7f2bd09 (2026-05-21) cache-line-aligns `Node (\#[repr(C, align(64))])` so the hot, frequently-pollled page-type tag does not false-share with the rest of the node, and moves the root index onto the same `SmartCell` machinery ordinary blocks use, off a separate `parking_lot` mutex. A later experiment (45cb048, 2026-05-28) that had range-query iterators hold a long-lived `BlockGuard` across a whole iteration was *reverted*: a 'static-lifetime guard held for an iterator's whole life is exactly the kind of long-lived, never-revalidated read that GC's reclaim bookkeeping does not know how to account for. The revert restores a bare `BlockRef`, re-borrowed fresh on each step, keeping GC's liveness accounting sound (§7).

5 Structural maintenance: splits, merges, and page capacity

Structure-modification operations (SMOs, `src/mv_tree/smo.rs`) decide, on the cold path, whether a page needs to split (overflow) or merge (underflow), and how.

5.1 Capacity tuning

34555e2 (2026-06-20) moved the internal-page overflow trigger from `max_keys()-3` to `max_keys()-1`: a page is now allowed to fill much closer to capacity before an SMO is attempted at all (**Bounded**). b29d347 is the 4 KiB-alignment trim already covered in §3 (**Measured**, 25%→100% aligned allocations, −9–12% dTLB load misses).

5.2 Split: choosing where to cut

`split()` decides between a **VERSION_SPLIT** (compact dead versions out of one page in place) and a **KEY_SPLIT** (cut at a key boundary). Two real bugs surfaced here under concurrent TPC-C stress in July–August 2026:

- 2a2607e (2026-08-02) — the **VERSION_SPLIT/KEY_SPLIT** decision used a strict `survivor\_count > capacity` test, so a page whose survivors landed at *exactly* capacity wrongly took the “just compact” path, producing an already-100%-full result with no room for the pending write — a panic on a root-leaf tree, an infinite retry otherwise. Fixed with `>=` (**Enabling**). This turned out to be the true root cause of a stress-test failure originally suspected to be hot-key version tearing (§6) — the failing page in fact held several distinct keys.
- dc30111 (2026-08-02) — the boundary search greedily combined “don’t tear a key’s version chain” and “fit within capacity” and could miss a valid boundary satisfying both. Fixed with a tiered search: (1) no tear and fits, (2) fits but may tear, (3) no tear but may overflow, (4) raw midpoint fallback (**Enabling**).

5.3 Merge: choosing what to combine

- dcb5abe — compares available room on *both* adjacent siblings instead of defaulting to the right one (**Bounded**).
- 4c01360 — dropped a `ts\_start`-based re-sort when combining two leaves: it risked reordering a live record ahead of a dead predecessor with a larger `ts\_start`, corrupting the backward-scan invariant from §3 (**Bounded**, and a correctness fix).
- 4bc78de — merging two leaves by `active\_count` alone could overflow `NUM\_RECORDS`, because `record\_survives\_gc` also keeps GC-protected dead records an active-count threshold cannot see. Fixed with an explicit `leaf\_merge\_would\_overflow` preflight that falls through to a key split when the true survivor count would not fit (**Enabling**).
- An experiment retrying an alternate merge candidate immediately (0659d34) was implemented and *reverted*: it spun without backoff and starved threads under contention.

5.4 A week of hardening (2026-07-31 – 08-02)

The fixes above were the tail of a dense, one-week debugging arc chasing two real TPC-C crashes: 7432525 hardened `InternalPage` overflow checks without yet fixing the crash; 5c54658 fixed both via “GC-gated survivor protection” (correctly distinguishing a genuinely committed delete from one whose abort-reversal still needs its predecessor intact) and corrected `bulk\_push` accounting (it had assumed every bulk-pushed record was active); 77a5689 reworked split/merge for a weakened fresh-node rule; fc8806b reverted a defensive `Result`-based workaround layered on `on\_overflow\_node/ on\_underflow\_node` once the real cause was actually found and fixed.

6 The self-overwrite optimization

This is the second load-bearing optimization singled out in §2: *when a single, still-uncommitted transaction updates the same key several times, overwrite the previous update in place instead of appending a new multiversion entry.*

6.1 The problem

Every write mints one more physical `RecordPoint` tuple, and an *uncommitted* tuple can be neither discarded (the transaction might still commit) nor coalesced (nothing has superseded it, since nothing else can even see it yet). The old update path was literally “delete old, push new” on every call, so one open transaction writing the same key N times left N physical, uncommitted, unreclaimable tuples for that one key. If N grows large relative to a page’s capacity, `split()`’s boundary search has no key boundary to cut at — its own documentation states it falls back to tearing the chain across two disjoint-fence siblings only when the page is entirely one key’s chain, silently orphaning the torn-off half from any fence-routed lookup. A concrete real-world trigger: TPC-C’s New-Order transaction pricing two order-lines for the same item lands two updates on the same `Stock` row inside one transaction.

6.2 The fix, as it exists in the current code

The logic now lives in free functions shared by both `DbTransaction` and `TpccTxn` (`src/mv_db/transaction.rs`, lines 92–216 as of this writing). The update path:

```
match leaf_page.latest_position(key, true) {
  Some(position) => {
    if tree.is_visible_stamp(worker_id, ts_start,
      leaf_page.version_at(position).insertion_stamp())
    {
      let stamp = TxStamp::new(worker_id, ts_start);
      tree.wal_log_write(stamp, |_| CRUDOperation::Update(key, payload.clone()));

      // Self-overwrite fast path -- see `insert_on_tree`'s doc.
      if leaf_page.version_at(position).insertion_stamp() == stamp {
        leaf_page.set_payload_at(position, payload);
        return (CRUDOperationResult::Updated(stamp.ts_start()), false);
      }

      // Otherwise: normal versioned update (delete old, push new).
      if !leaf_page.version_mut_at(position).delete(stamp) {
        return (CRUDOperationResult::ZeroAffected(KeyAlreadyDeleted), false);
      }
      let current_len = leaf_page.len();
      leaf_page.push_uncommitted(
        RecordPoint::new(key, VersionInfo::new(stamp), payload), current_len);
      leaf_page.commit_delta(0, 1);
      (CRUDOperationResult::Updated(stamp.ts_start()), true)
    } else { (CRUDOperationResult::Conflict, false) }
  }
  None => (CRUDOperationResult::ZeroAffected(KeyDoesNotExist), false),
}
```

Detection is a single equality check, not a heuristic: a `TxStamp` packs (`worker\_id, ts\_start`) and is constant for the *whole* transaction, so “was the newest live version of this key written by me, in this very transaction” reduces to `insertion\_stamp()==stamp` — an OSIC concept, as §2 points out. **Safety** follows from visibility: `is_visible_stamp` only returns true for the same worker or a stamp already in the commit log; neither applies to any other transaction while this one is open, so there is no possible reader relying on this specific uncommitted tuple’s current content. If the found record belongs to another transaction, or is an already-committed version, the code falls through to the ordinary delete-then-push path unchanged.

The boolean second return value (`false` on the fast path) tells the caller whether a new **written-set** entry is needed: mutating an already-open transaction’s own uncommitted record in place mints no new physical version, so the entry an earlier write already pushed for this key still

covers abort.

6.3 Delete/reinsert cycles use the same bound

`insert_on_tree` (lines 92–151) applies the symmetric case: the first `delete(key)` followed by `insert(key,value)` in one transaction must append a replacement tuple (the committed predecessor must stay physically present for abort’s undelete). But repeating that pair in the same transaction used to append another replacement every cycle. The current code finds the newest non-invalid record and, if its insertion *and* deletion stamps both equal the current transaction’s stamp, clears the deletion mark and replaces the payload in place instead of appending — deliberately without adding another write-set entry, since the first replacement’s entry already makes abort invalidate that tuple and undelete the original predecessor. N delete/reinsert cycles now create one replacement tuple; commit exposes only the final payload; abort restores the pre-transaction value; WAL recovery replays the final logical state.

6.4 The chronological story

Two mechanisms are easy to conflate; only the second is the trick above.

An older, unrelated mechanism: GC-driven update-in-place (2025–2026). `bea1117` (2025-06-05) added an update-in-place optimization to the *non-transactional*, single-operation CRUD dispatch path (`src/mv_query/dispatch.rs`): if a record’s insert version predates every live snapshot — GC has already proven nobody can need the old *committed* value — its payload is overwritten in place rather than versioned again. It is gated off whenever a WAL is attached (the shortcut cannot be represented as a single WAL record) and remains in the codebase, but it is a GC/version-bloat heuristic for single-op writes, unrelated to the transactional trick above.

The actual fix (August 2026). `160186e` (2026-08-01) first found that `abort()` only reverted the *newest* of several same-key writes, leaving older self-written versions permanently visible; fixed to unwind the whole chain one physical version at a time. `85ac1b0` (2026-08-01) is the fix itself, framed exactly via the TPC-C Stock-row scenario; its commit message also documents a *reverted* companion idea — lowering the leaf overflow trigger to buy compaction more headroom — abandoned because it caused a hang under concurrent stress. `fbedd74` added a minimal repro isolating a *different* contributor to the same class of pressure (a long-open snapshot keeping an old committed version GC-protected while other transactions keep committing new versions of the same hot key — no same-transaction repeats involved; the repro’s own documentation states self-overwrite “cannot help here”). Tracing that repro further (`2a2607e`) found the real cause was the split off-by-one from §5, not hot-key tearing at all.

6.5 Resulting bound, and a known residual

N updates or delete/reinsert cycles to one key by one transaction now cost at most one unresolved replacement tuple. First-writer-wins prevents another transaction from concurrently extending that same unresolved chain. Once the writer resolves, ordinary GC compaction discards obsolete committed versions; pages with multiple keys remain splittable at a key boundary. The known, documented residual: writing the same key N times in one transaction still leaves *one* extra physical entry versus never writing it at all — self-overwrite bounds the growth to $O(1)$, it does not eliminate it. The TPC-C stress invariant still permits a narrow 0.5% difference between aggregate `Stock` counters and inserted `OrderLine` rows, tracked as a separate, still-open accounting discrepancy unrelated to same-key splitting or reader retention.

7 Garbage collection: reclaiming dead blocks without contention

7.1 From a global mutex to sharded, lock-free reclaim

acdc7a8 (2024-02-14) is the first GC implementation: a global `Mutex<RBTREE<SnapShot>>` of active transactions and a global `Mutex<LinkedList<(*mutVersion,BlockRef)>>` of dead pages, with the commit message already naming the problem that would dominate the next several months: “locks must be left out, or possible deadlocks appear!” 2e3ad21 switched allocation to `try_lock()` on both structures (bail out rather than block, **Enabling**); 95e149f moved the active-tx index to the project’s own OLC-locked concurrent B+-tree (itself later superseded). The current end state (`src/mv_gc/block_tracer.rs`) drives liveness directly off `TxContext::live_min_snapshot()` (§9) rather than a separate active-tx structure, and reclaim off a worker-sharded `SkipMap` rather than one list or tree.

7.2 Two ThreadSanitizer races, closed structurally

- d0e0977 (2026-07-21) — a separate `retired:AtomicBool` field was folded into the Smart-Cell’s own `cell_version` bits (§4). Before this, a two-step “mark retired, then let `Drop` unlock later” pattern raced a concurrent reuse’s `clear_retired()` against the dying guard’s deferred `Drop` storing a value computed at lock-acquisition time, which could silently revert a freshly reused cell’s version — confirmed empirically as hangs/livelocks under GC plus heavy concurrency.
- ecc9900 (2026-07-27) — two independent bugs. First, the dead-block index was keyed by a bare `Version`, but a merge’s two dying siblings deliberately share one death version (the replacement block’s birth version); `SkipMap::insert` on a duplicate key silently displaces the existing entry, leaking one of the two retired blocks. Fixed by keying on `(Version, block\_address)`. Second, `insert\_stamp/delete\_stamp` were plain, non-atomic fields mutated by `delete/invalidate` while lock-free OLC readers read the same word with no lock — a genuine torn-read race, confirmed by ThreadSanitizer. Fixed by backing both with `AtomicU64/Relaxed` (sufficient because the hazard was torn reads, not ordering).
- fe03d90 (2026-07-28) — the capstone: `version\_region` moves from `AtomicVersion` to plain `Version`, and the in-place `mark\_version\_obsolete fetch\_or` (itself confirmed racing range-query iteration) is deleted outright, replaced by the `live\_mask()` derivation described in §3. No writer ever mutates a slot after publication anymore, so the race does not merely get fenced correctly — it stops being possible.

Recurring pattern. Nearly every “racey”/“stabilize” fix in this section follows the same shape: something was mutated *in place* on an already-published, lock-free-readable slot, requiring ever more careful atomic/fence discipline to patch around it, until the design changed so nothing is mutated after publication at all. This mirrors §4’s fence-removal trajectory: converging from “atomics and fences bolted on to patch races” toward “structure the data so the race is structurally impossible.”

7.3 Sharded reclaim, historical routing, and abort-safe retention

`block_tracer.rs` holds one `SkipMap` shard per worker (§9 covers the mechanism); a nonblocking reuse fallback allocates fresh memory rather than joining the page-latch dependency graph when reuse metadata is contended. Old readers are kept on their retired *source* block via historical root/version routing rather than having every old-reader-visible record copied forward into replacement pages during a rebuild — copying for the globally oldest reader would couple every key’s reclaim to one transaction and risks a self-reinforcing compaction livelock (a version of exactly this failure mode is discussed as an open issue in §15).

8 Root indexing strategies

A reader must find “the root as of snapshot S ” before traversing anything. `RootIndexType` selects one of four version-keyed structures:

Strategy	Structure	Lookup	Notes
Vanilla	<code>LinkedList<Root<..>></code>	$O(n)$ linear	Simplest baseline.
SK (SkipList)	<code>crossbeam_skiplist::SkipMap</code>	$O(\log n)$ expected	ex- Lock-free concurrent inserts/lookups.
DexaBTree	external OLC-locked B+-tree, keyed by <code>Snapshot</code>	$O(\log n)$	Reuses a full concurrent B+-tree just to index root pointers.
FrugalList (default)	lock-free singly-linked list with a probabilistic skip-tower ($p = 0.5$)	sub-linear expected	ex- Appends are one <code>AtomicPtr</code> store (no CAS loop needed — already serialized by the enclosing write latch); much smaller per-node footprint.

These are not partitioned by version range — each is one tree-wide structure holding every historical root. They optimize a different axis than §9: the cost of finding the root pointer itself (a potential single hot spot every reader must consult), trading lookup complexity against per-append cost and memory overhead.

9 Sharding as a system-wide pattern

The word “sharding” appears in four genuinely distinct places, none of them classic horizontal key-range partitioning of one index. Each instead partitions a specific hot, shared piece of *bookkeeping state* by worker (or, in one case, by table). §2 argued that this pattern presupposes both OLC (so there is lock-free contention worth removing) and a stable per-worker identity (so there is something to shard along); this section is the pattern applied four times.

9.1 Tables as shards: one tree per table

5f92e9f (2026-07-15) — before this, multiple logical tables were multiplexed into a single tree. `Database` now holds `tables:ArcSwap<TableList<..>>`: each table gets its own tree, root index, and — importantly for §7 — its own dead-block tracker. Concurrent transactions touching different tables no longer contend on one tree’s root/internal latches or one dead-block skip list. What *must* stay global for cross-table snapshot isolation — the clock, commit-log pruning, snapshot liveness — moves into one shared, non-generic `TxContext` introduced in the same commit (**Bounded**).

9.2 GC dead-block tracker: own-shard-first, round-robin steal

`BlockTrace` holds one `SkipMap` shard per CPU (`shard\_for(worker\_id)=worker\_id\%shards.len()`). Registering a died page only ever inserts into the dying worker’s own shard; reclaim pops from the caller’s own shard first, falling back to a shared round-robin cursor across other shards only on a miss. **Why:** death versions all come from one global clock, so before sharding, concurrent SMOs from every worker inserted into a *single* skip list clustering near its current maximum key — an always-contended tail. Since a reclaimer only needs *a* reclaimable block, not the globally oldest one, trading global pop-order precision for per-worker insert locality is

sound. f9fc960 (2026-07-23) added the “try own shard first” fast path (before it, reclaim always scanned round-robin even when the caller’s own shard already had a usable entry).

9.3 TxContext: cache-padded, per-worker atomic slots

Three parallel vectors, one slot per `WorkerId`: `live_tx` (each worker’s currently published outermost snapshot), `live_tx_depth` (reentrancy depth, so only the outermost `begin_snapshot` touches `live_tx`), and `in_flight_bound` (a conservative lower bound published while a worker is mid-registration — has drawn a `ts\_start` but not yet inserted). Each worker only ever writes its own slot; `free_block/live_min_snapshot` scan all slots with `Acquire` loads, $O(\max_workers)$, no cross-slot synchronization. This directly replaced two shared, globally-contended structures, and the commit history records the profiling numbers that motivated it: `live_tx` replaced a shared `SkipMap<Snapshot, AtomicUsize>` measured (per an in-code comment) eating “roughly a quarter to half of all CPU cycles” on a saturated TPC-C run; `in_flight_bound` replaced a single global `AtomicUsize` that, under a 16-thread update-heavy YCSB workload, rejected 73.8% of `free_block` attempts (versus 5.2% at 2 threads). eac6e40 (2026-08-02) then wrapped all three vectors in `crossbeam\_utils::CachePadded`: previously up to 8 adjacent workers’ slots could share one 64-byte cache line, directly contradicting those fields’ own design-intent comments — a pure layout fix targeted at 64-core/128-thread scale, where cross-CCD coherence traffic is expensive.

9.4 Thread-local worker identity and snapshot cache

`src/mv_sync/worker.rs` hands out a `WorkerId` once per (tree, thread), cached thread-locally keyed by a process-wide `tree\_uid`. A second thread-local, `SNAPSHOT\_CACHE`, gives each worker its own `SnapshotCache` — the OSIC paper’s “thread-local snapshot cache” — rather than a `Vec` on the tree indexed by `WorkerId`: using actual thread-local storage means no thread ever *could* touch another worker’s cache, stronger than “happens to always pass its own id.” A dedicated regression test file (`tests/todays\_optimization\_regression\_tests.rs`) verifies, among other things, that a thread cycling through three exactly-sized tree registries never loses a `WorkerId` across the two inline TLS slots’ lazy overflow path.

10 OSIC integration: instant commit, lazily hardened

10.1 What OSIC is, and how cMVBT realizes it

Ordered Snapshot Instant Commit [3] decouples visibility from durability. Every transaction draws two timestamps from one global, monotonically increasing counter (`GlobalClock`, a single `AtomicVersion`, `fetch\_add(1, SeqCst)`): `ts\_start` at begin, `ts\_commit` at commit. Each worker keeps its own append-only `CommitLog` (`Mutex<Vec<Version>>`); `commit()` draws `ts\_commit` then pushes it under the lock — that ordering is exactly what prevents a reader from observing a reserved-but-not-yet-inserted stamp. Visibility (`is_visible`) rejects invalid (aborted) stamps outright; same-worker writes are visible if `stamp.ts\_start() <= reader\_ts\_start`; otherwise visibility is `LCB(stamp.worker\_id, reader\_ts\_start) > stamp.ts\_start()`, computed via each thread’s own `SnapshotCache` memo. `WalWriter` is a lock-free MPSC channel to a background flush thread: `log\_with\_stamp/log\_commit` encode and enqueue, returning immediately — fire-and-forget, no per-op `fsync` wait. `flush\_loop` sleeps a short `GROUP\_COMMIT\_LINGER` (200 μ s) after the first message, drains everything else non-blockingly, batches it into one `write\_all + sync\_data()`, and advances a `hardened` watermark to the maximum `ts\_start` of any `Commit` message in that batch. Recovery keeps only writes with a matching commit marker (commit-gated replay), sorted by (`ts\_commit`, `fileorder`) since commit order can differ from log order across concurrent workers. `DbTransaction::`

`commit` appends to the committing worker’s `CommitLog` — making every write visible everywhere, instantly — then logs exactly one fire-and-forget WAL commit marker. *This is the whole “instant commit”*: one in-memory append, with nothing on the write/commit path waiting on disk I/O; durability is a separate, batched, background concern, exposed only via an explicit `wait\_wal\_hardened()` API for callers who specifically need a durability checkpoint.

Evolution. A pre-OSIC era (Dec 2025 – Jan 2026) used a hand-rolled sharded thread-indexed “committed” watermark array — in effect an imperfect reinvention of the `CommitLog/GLC` pair, with a sequence of patches fixing edge cases around threads changing roles or exiting, reading as a slow discovery, by trial and error, of exactly the problem OSIC’s design solves cleanly. 714a82f (2026-07-10) replaced it wholesale with the paper’s actual design.

10.2 The WAL performance campaign

A short-test profiling pass (`scripts/compare_engines.py`, `perfrecord/` `perfannotate` on a debug-symbol release build) found three concrete issues, fixed together and re-measured on the same harness invocation.

1. `target-cpu=native` was silently dead. `Cargo.toml` had a `[build]rustflags` block, but Cargo only reads `rustflags` from `.cargo/config.toml`, never the manifest — the whole binary had been compiling to the generic x86-64 baseline. Fixed by moving the flag into a new `.cargo/config.toml` (**Enabling**, confirmed present in the current repository).

2. Hand-rolled CRC32 was the single biggest hotspot. A byte-at-a-time, 8-shift-per-byte bit loop was **22.6% of all CPU cycles** (self time) on YCSB-A at 64 threads. Replaced with a 256-entry lookup-table CRC32, same polynomial/format, pinned against the standard “123456789” → 0xCBF43926 check value. Self-time dropped to 11.3% on the same workload.

3. The single shared WAL channel was a real contention point. `Sender::send` showed 86% of its own samples in a `crossbeam-channel` internal spin-wait, worth ~8% of *all* CPU cycles on TPC-C at 64 threads. Fixed by splitting `WalWriter` into `NUM\_SHARDS=4` independent (channel, flush-thread) pairs keyed by `worker\_id\%4`, all still writing through one shared `Arc<Mutex<File>>` so on-disk format and recovery are unaffected; the shared `hardened` watermark switched from `store` to `fetch\_max` since multiple flush threads can now finish out of order.

Workload	Threads	Before	After (Measured)
tpcc	16	40,902	43,913 (+7.4%)
tpcc	64	57,667	67,812 (+ 17.6%)
ycsb_a (50% update, Zipfian)	16	1,603,135	2,010,940 (+25.4%)
ycsb_a	64	1,339,295	2,258,954 (+ 68.7%)
ycsb_e (scan-heavy)	16	2,137,315	2,483,067 (+16.2%)
ycsb_e	64	1,789,143	2,679,747 (+ 49.8%)
ycsb_f (read-modify-write)	64	1,355,666	2,305,318 (+ 70.1%)

The read-only/read-mostly workloads (ycsb_b, ycsb_c) moved by under 1.2% either way, confirming the fixes are surgical to the write path. The clearest structural result: the two workloads that previously got *worse* with more concurrency (ycsb_a, ycsb_e) now scale up instead of down. Not addressed by this pass: TPC-C’s OCC conflict rate remained very high at 8 warehouses/64 terminals (Payment ~71–76% of attempts conflict and are discarded; New-Order ~47%) — flagged in §15.

10.3 A lock-free WAL backend, and two concurrency bugs it took to land

A second, concurrent line of work attacked the same contention differently: `LockFreeWalWriter/LockFreeWalB` removes the channel and dedicated writer thread entirely — every worker reserves its own byte range via `tail.fetch\add` and calls `pwrite` directly, optionally batching its own records locally (`LocalBatch`) before flushing. `WalBackend` wraps both designs behind one enum so callers pick either at attach time.

Merging this (ebdd789) with the sharded-channel work (4a1a50a) surfaced two bugs unrelated to either design’s core idea:

- **An OOM, not a WAL bug.** `mv\_test::RESTART\_TRACE` (a write-restart diagnostic, documented “off by default”) had been left `true` in checked-in code. Every OCC restart recorded a `String` key into a process-lifetime global map that nothing ever clears; a comparison loop calling `run\_tpcc()` six times in one process accumulated across every iteration until the fastest config grew past 12 GB RSS and got OOM-killed, taking the terminal session with it. Fixed by restoring the documented default and adding `reset\_restart\_trace()`, called at the start of every `run\_tpcc()` (verified present in the current source, §16).
- **A cross-shard `hardened\_version` race.** The sharded `WalWriter` published every shard’s own confirmed watermark into one value via `fetch\_max` — correct only if every shard’s watermark were directly comparable, which it is not, since `ts\_start` is one global sequence spread across shards unpredictably. One busy shard’s flush could advance the shared watermark past a `ts\_start` a slower shard had not flushed yet; a consistency test caught it directly, a quarter of written records missing from the reconstructed WAL. A first fix attempt (minimum of every shard’s raw confirmed value) traded the false-positive for a hang: a shard whose submissions never reach some other shard’s higher watermark would never satisfy a plain minimum, even once fully caught up. The actual fix (9ced1a3) gives each shard a `submitted` watermark alongside `confirmed`, and treats a shard as imposing no constraint once `confirmed >= submitted` (fully drained) rather than comparing raw values across shards. Verified: the failing test 5× clean, the full suite 3× clean.

Synthetic backend comparison (original report, debug build):

Backend	@8 threads	@16 threads
<code>WalWriter</code> (4-way sharded)	1,006,755 ops/s	1,353,816 ops/s
<code>LockFreeWalWriter</code> , unbatched	752,179 ops/s	748,676 ops/s
<code>LockFreeWalWriter</code> + <code>LocalBatch(64)</code>	2,549,030 ops/s	5,089,502 ops/s

Unbatched lock-free writes lose to the sharded channel past a few threads: each write pays a full `pwrite` syscall plus filesystem-level inode-extension serialization on a shared growing file. Per-thread batching is what actually wins. §16 reproduces this exact comparison, in release mode with LTO, with a fuller thread/batch sweep, on the machine this document was written on.

10.4 Ticket-less logging and exact framing hints

A follow-up `perfrecord` pass, once the channel contention above was no longer dominant, found two purely allocator-side hotspots it had been masking: `WalWriter::enqueue` built a fresh `bounded(1)` ack channel on *every* call to return a flush ticket that every production call site always discarded; and every framing buffer’s capacity hint (24–44 bytes) was tuned for the module’s own `u64`-payload tests, while `YcsbRow` is roughly 25× that and `TpccRow` variants routinely 7–10× over, forcing several grow-and-copy reallocations per WAL write. Fixed by splitting log-

ging methods into a ticket-less default plus an explicit `*\_with\_ticket` variant, and by adding `WalPayload::wal\_encode\_size\_hint()` with exact overrides mirroring each payload type’s own encoder.

Metric	Before	After	Change
TPC-C, WAL on (tpmC)	2,675,157	3,013,193	+12.6%
YCSB-A, WAL on (ops/sec)	1,374,921	1,723,455	+25.3%
Synthetic WalWriter, 16 threads, u64 payload	1,353,816 ops/s	2,427,071 ops/s	+79%

The synthetic case gained the most because, against an 8-byte payload, the discarded ticket’s channel allocation was close to the entire per-write cost. **Not fully closed:** a fresh ~ 1000 -byte `Vec` is still allocated per WAL write on `WalWriter`’s cross-thread path; a true buffer pool was flagged as a follow-up and remains unimplemented (§15).

10.5 Two dead ends, honestly reported

A **thread-local scratch buffer on `LockFreeWalWriter`** was implemented and verified correct, on the reasoning that the lock-free writer’s synchronous `pwrite` has no cross-thread ownership problem. Tracing the actual call graph found it was dead weight: every real caller goes through `LockFreeWalBackend`, which always routes into `LocalBatch`’s own persistent buffer first, even at `batch\_size:1` — so only this module’s own unit tests and the deliberately unbatched benchmark arm ever reached the patched method. **Reverted** rather than kept with no real effect.

An **opt-in `mimalloc` allocator feature** targeted the genuine cross-thread free pattern in `WalWriter` (one thread mallocs the framed buffer, the flush thread frees it). Measured end to end: YCSB-A gained 2.3–3.0% on the one backend with that cross-thread free, a wash on backends without it; TPC-C *lost* 3.3–4.9% across every backend, including ones with nothing to do with this allocation pattern, all six (backend $\times$ terminal-count) combinations moving the same direction. **Not adopted as default** (`jemalloc` stays the global allocator); kept as an opt-in feature since the trade-off is genuinely workload-dependent, not a universal win.

10.6 Atomic operations: an auto-commit fast path alongside real transactions

A single-key *atomic* operation is auto-committed, not a snapshot transaction with a one-entry write set. Its write path: take a short reclamation pin during traversal, acquire the target leaf’s exclusive OLC writer guard, verify the newest physical version is visible at the current OSIC boundary, draw the stamp and log the WAL write, mutate the leaf while unobservable behind the guard, append to the worker’s commit log, *then* release the guard — publishing an already-committed version. The leaf-guard release is the linearization point for other OLC traversals; by the first instant a reader can validate the modified leaf, the new record’s stamp is already in the commit log, closing the “published leaf entry, commit log not updated yet” window a naive design could otherwise expose. Deliberately, an atomic write does *not* call `begin_snapshot`: it wants the newest committed state rather than a historical view spanning several statements, and it retains no root/page/timestamp after the call, so registering a full long-lived snapshot would add clock-tick and bookkeeping cost the protocol does not need. The YCSB driver exposes both modes (`--cmvbt-ycsb-modeatomic|transaction`) without changing record shape, key distribution, or WAL settings, specifically so the cost of the transaction machinery itself is measurable rather than baked into every number.

10.7 Known, documented limitations

Unbounded `CommitLog` growth under long GC-off historic-query runs (a chunked/segmented log was considered, explicitly deferred). `MAX_WORKERS_CAP=256`, a generous but hard ceiling. Abort must revert writes in strict LIFO order — a documented but delicate invariant, not a generically solved problem. The self-overwrite residual noted in §6. And `ecc9900`'s torn-read fix is explicit in its own documentation about closing one specific hazard, not every conceivable concurrent-mutation hazard on the lock-free OLC read path.

11 Range scans and analytical terminals

11.1 Why internal pages cannot be scanned in physical order

An internal page is append/version ordered, not key ordered: structural modifications append replacement child entries while retaining historical entries older snapshots still need. Neither walking physical slots left to right nor sorting every snapshot-matched slot produces a correct scan — the latter includes superseded, overlapping children. `RangeQueryIter` instead retains internal pages on its current path: at each level it searches entries in reverse append order and selects the newest snapshot-visible fence containing the scan cursor, descends, and after consuming a leaf advances the cursor one key past that leaf's fence and reroutes from the retained parent — the same selection rule as point lookup, allocating no child lists. An exact full-domain scan (`Key::MIN..=Key::MAX`) is detected once before the leaf loop and uses a visibility-only record loop, skipping two bound comparisons per physical record that every bounded interval still needs. The final leaf is a special case: key increment saturates at the maximum key, so both the materialized and streaming paths must explicitly complete when the consumed fence reaches the requested upper bound or the tree maximum, or a full-range streaming scan would route the unchanged maximum cursor back to the final child forever.

11.2 Rejected: child-list reuse, small-page dedup, hybrid traversal, lean frames

Four related optimizations were implemented, benchmarked (an experimental version appeared 18–20% faster), and then **removed** once GC stress testing proved their shared premise incorrect: reusing an eagerly constructed child-list snapshot, deduplicating boundaries by hashing/linear scan, a hybrid narrow-range traversal built on that same list, and releasing an internal page early after expansion. Equal-boundary deduplication cannot prove that arbitrary historical intervals are fully superseded; under copy-on-write GC it let both a current route and an overlapping historical route be scanned, returning duplicate logical keys. The active cursor-based traversal above was already allocation-free and always selects the newest visible route, making the rejected machinery unnecessary as well as unsound.

11.3 Streaming terminals: zero-copy analytical consumption

`RangeQueryIter::for_each_ref/fold_ref/try_for_each_ref/ count_ref` call a visitor with (`Key, \&Payload`) directly, avoiding one `RecordPointResult` construction, one payload-handle clone, a retained leaf buffer, and materializing a whole result vector, per match. The payload reference is callback-scoped and must not be retained. `DbTransaction` exposes the same four operations (`range\count`, `range\fold`, `range\for\each`, `try\range\for\each`), keeping its function-local table `Arc` alive until the terminal operation consumes the scan; `TpccTxn`'s versions dispatch across both standard and big-tree table classes (§13) with identical behavior.

Measured (200,000 shuffled u64 rows, five warmups, 40 alternating samples): streaming was $1.165\times$ – $1.173\times$ faster than the materialized iterator for a full scan ($\sim 16.8\%$; 28.8–29.3 M rows/s vs. 24.7–25.1 M rows/s). A later, separately measured `count\_ref` build reached **49–53 M**

rows/s on full scans and showed $1.18\times$ – $1.37\times$ speedups on both full and narrow-range scans; these routing-adjacent numbers are retained only as design history, since the specific implementation measured there (inline reusable scratch, a single-child fast path) was the one later reverted for correctness in the previous subsection. `count\_ref` itself remains enabled and its own number is not affected by that revert.

11.4 Fix 1: test the range before the visibility check

`RangeQueryIter::refill` and `key_range_read_from_root` both filtered each physical record with `r.version().matches(is\_visible)\&\&range.contains(r.key())`. `VersionInfo::matches` calls `is\_visible` up to twice per record (insertion and deletion stamp), on *every* dead/super-seded record a leaf holds — the majority of a garbage-heavy leaf’s contents, per §13’s scan-cost findings — and it did this *before* the cheap, inlinable range comparison ever got a chance to reject the record. Swapped the order in both places: pure reorder of an existing `\&\&`, zero behavior change. No benefit for a full-table scan (its range always contains every key). Real benefit for a *narrow* scan sharing a leaf with keys outside its range — e.g. CH-benCHmark Q4/Q5’s per-order `OrderLine` sub-scans. **Measured** (8 warehouses/16 terminals, 20s runs, noisy/contented, indicative not averaged): Q4 latency ~ 614 – 617 ms $\rightarrow$ ~ 505 – 507 ms (**$\sim 18\%$ faster**); Q5 ~ 6.81 – 6.90 s $\rightarrow$ ~ 6.09 s (**~ 11 – 13% faster**). Q1/Q6 (full-table scans) showed comparable-or-noisy numbers either way, exactly matching the mechanism’s prediction.

11.5 Fix 2: remove the dyn FnMut indirection

Reordering only helps *out-of-range* records; every record actually in range still paid for a non-inlinable indirect call on every `matches` invocation. `VersionInfo::matches` was generalized from `\&mutdynFnMut(TxStamp)->bool` to `<F:FnMut(TxStamp)->bool+?Sized>(\&mutF)` — backward compatible, since `dynFnMut(..)` itself satisfies the new bound, so every existing caller keeps compiling with zero behavior change. `RangeQueryIter::refill` was restructured to call a new `with\_snapshot\_cache\_and\_logs` accessor once per `refill()` (not once per leaf), build a concrete `is\_visible` closure inline, and use it directly — no `dyn` anywhere in this path. `query.rs`’s two call sites were left completely untouched.

A methodological scare, worth recording as a case study. The first verification pass looked like a real regression: `verify\_concurrent\_shared\_keys`’s “never loses an update” stress test flaked 2/5 then 4/20 with the change, against a first baseline sample of a clean 10/10. That pattern — a concurrency test flaking only once concurrency-adjacent code changes — is exactly what a real bug from this kind of change would look like. Re-investigating rather than reverting on the spot: a *wider* baseline sample, drawn after the same session’s own repeated heavy test/build load had warmed the machine up, came back 2/19 ($\sim 10.5\%$) on the identical test and panic line — the earlier clean 10/10 baseline had simply been drawn before the machine warmed up, not because the flake was not there. A clean, side-by-side 20-run batch with the change applied came back 2/20 (10%), statistically indistinguishable from baseline. **Lesson:** do not trust a 5–10 run sample of a timed concurrency test, in either direction, on a machine that has been running heavy benchmarks for a while — draw a same-conditions baseline of comparable size before attributing a failure to a change. (This exact flake, and its true underlying cause, are followed further in §15.)

11.6 The point-read miss bug this work incidentally surfaced

Investigating the flake above led to a second, independent, and more serious bug, now fixed and committed (c0fe172): `key_point_read_from_root`’s `skip_while` and `key_range_read_from_root`’s binary-search cutoff both assumed a leaf’s physical append order tracks `ts_start` order. That is false under this system’s concurrency model — `ts_start` is drawn at `begin()`,

before the writer acquires the leaf’s write lock to physically append, so two concurrent writers can draw `ts\_start` in one order but append in the other. When they diverged, the point-read shortcut could walk straight past the one visible record and return an empty result for a live, never-deleted key. **Fix:** removed both shortcuts; always scan the whole leaf — the same cost `RangeQueryIter::refill` already always pays, so no new asymptotic cost class is introduced (**Enabling**). Verified with 20+20 clean runs of both affected tests, versus the $\sim 10\%$ baseline failure rate documented beforehand.

12 Workload and benchmark-harness optimizations

Measurements in this section used a release build, 200,000 preloaded rows, four workers, a three-second timed phase, Zipf $\theta = 0.99$, GC enabled, no WAL, and the frugal root index — they measure a combined batch, not isolated attribution to one row.

Change	Implementation and gain	Evidence
Bulk-generate YCSB values	Rows allocate their final payload once, filled in place from buffered random bytes. Full-row YCSB-A: 2.52M $\rightarrow$ 3.11M ops/s (+ 23.5%).	Measured
<code>writeallfields</code> (standard mode)	Standard YCSB updates one field via a leaf-latched <code>update_with</code> patch, not a full-row regenerate. Standard single-field YCSB-A: 4.81M ops/s (+90.8% vs. the old full-row baseline — different semantics, not a fair like-for-like number).	Measured
<code>point_exists_si</code>	Hit/miss-only reads test visibility in place, no result vector or payload clone. Combined with alias Zipf sampling, YCSB-C: 7.16M $\rightarrow$ 14.82M ops/s (+ 106.8%).	Measured
Walker-alias Zipf sampling	A finite alias table built once at setup; sampling is two random draws and a lookup, replacing per-request floating-point power computation. $O(n)$ setup, 8 bytes/key (~ 160 MB at 20M keys).	part of above
Empty read-only commit elision	A read-only transaction with an empty write set releases its snapshot without drawing <code>ts_commit</code> or appending a commit-log entry — one clock tick and one commit-log op removed per such transaction.	Bounded
Sampled YCSB-E scan latency	One in 1,024 scans receives a latency timestamp; the timeseries clock refreshes every 256 ops. YCSB-E: 4.59M $\rightarrow$ 6.22M ops/s (+ 35.4%).	Measured
Full-domain scan specialization	<code>try_for_each_ref</code> detects <code>Key::MIN</code> . <code>.=Key::MAX</code> once, uses a visibility-only leaf loop (see §11).	included above
Isolate CH Q1/Q6	<code>OlapMode::ChQ1/ChQ6</code> run only the named query; the old rotating <code>ch</code> mode mixed in unrelated Q4/Q5 joins.	Enabling

Also unrelated to raw throughput but worth recording: Zipfian rank scrambling (FNV-based)

was added specifically to make the benchmark’s key-access locality *more* representative of real Zipfian workloads, not to make the engine faster — and **fastrand** replaced the earlier RNG for TPC-C parameter generation purely to take generation cost off the transaction hot path. Both changed the generated distribution, so before/after engine comparisons using either need to pin the generator version. Finally, the **RESTART_TRACE** OOM story from §10 directly motivated two always-on (not `\#[ignore]`d), deliberately tiny WAL perf smoke tests, and the default TPC-C/YCSB comparison scale was reduced to a few seconds per point specifically so the full suite (151 tests as of this writing, §16) runs on an ordinary development machine in single-digit seconds; **CMVBT_FULL_BENCH=1** restores the original large-scale configuration for dedicated performance runs.

13 Case study: **BigTreeSize** — a contention/garbage continuum, worked end to end

This section is presented as a standalone case study because it is the clearest example in the whole codebase of an optimization whose two extremes are each bounded by a *different, independently mechanistic* cost, and because the investigation of it produced and then caught its own benchmark methodology bug — a useful lesson in its own right.

13.1 Motivation

Per-table root-restart attribution (`mv\_test::dump\_root\_restarts\_by\_table`) found that **district** and **warehouse** alone accounted for **~99.97% of all root restarts** at default TPC-C scale, despite being the smallest tables in the schema (10 and 1 row per warehouse, respectively). The reason: leaf overflow triggers at a hard 100%-full ceiling with no tunable slack, and these two tables are hit by write volume disproportionate to their row count, so their physical version-chain growth crosses that ceiling roughly every 40 writes, forcing a compaction that write-locks the one page nearly every concurrent transaction routes through.

13.2 The current design: an eight-variant size-class enum

BigTreeSize (`src/mv_bench/tpcc_schema.rs`) gives `Table::Warehouse/ Table::District` their own tree type (`TreeClass`, plain tag dispatch, no `dynTrait`), independent of the base tree’s own untouched 123-record leaf. As of the current code, this is an eight-way, not five-way, continuum, spanning both below and above the base tree’s own size, each variant’s `NUM\_RECORDS` chosen so `OptCell<Block<..>>` lands exactly on a page-size multiple with zero waste:

Variant	Records/leaf	Page size	Measured trade-off (in-code, documented)
KiB1	27	1 KiB	Smallest variant confirmed to hold up under concurrent load — see below for why nothing smaller exists.
KiB2	59	2 KiB	Same shape as the untouched base tree, built as its own <code>TreeClass::Big</code> instance — the “no augmentation” reference point.
KiB4	123	4 KiB	
KiB8	251	8 KiB	Only $\sim 1.2\text{--}1.3\times$ root-restart reduction — barely more than off.
KiB16	507	16 KiB	$\sim 2\times/\sim 1.6\times$ reduction (district/warehouse).
KiB32 (default)	1019	32 KiB	OLAP scan throughput fully recovered ($\sim 100\%$ of the untouched-leaf baseline) while root restarts are still down $\sim 4.4\times/\sim 2.6\times$ — the measured sweet spot.
KiB64	2043	64 KiB	$\sim 8\times/\sim 4.6\times$ reduction, at a real but moderate OLAP scan cost ($\sim 82\%$ of baseline).
KiB512	16379	512 KiB	Maximum measured contention reduction ($\sim 32\times/\sim 18\times$), at a steep OLAP scan cost ($\sim 21\%$ of baseline).

The upper bound’s mechanism is unavoidable, not a tuning oversight: a leaf’s `overflow\_records\_count()==NUM\_RECORDS` ceiling is also *exactly when a leaf’s dead/superseded versions get compacted away*, and `RangeQueryIter` scans a leaf’s entire physical record array, live and dead alike. A bigger `NUM\_RECORDS` therefore means more accumulated dead-version garbage sits in the leaf between compactions, and every range scan over that table pays for wading through it. There is no way to decouple “avoid frequent compactions” from “keep dead-version garbage bounded” in this design — they are the same operation, which is exactly why `BigTreeSize` exists as an explicit, chosen point rather than a single hand-picked number.

The lower bound is a proven, not merely observed, floor. Every split-push into an internal node *appends* two fresh entries rather than overwriting (`on\_overflow\_node`); the superseded entry goes dead until GC compacts it. A freshly split node (two live children) needs `FAN\_OUT>=4` just to have room for one further push, and each subsequent push costs two more raw slots against a budget of `FAN\_OUT-1`. At `FAN\_OUT=3` that margin is exactly zero: a fresh root sits at the overflow threshold permanently, before it can accept even one push — a genuine, concurrency-*independent* deadlock, confirmed by tracing through `lacks\_room\_for\_split\_entries/on\_overflow\_node/unsafe\_degree\_root`. Larger-but-still-small `FAN\_OUT` compiles and is not provably deadlocked the same way, but empirically degrades from “fine” to “indistinguishable from hung” as concurrency rises: `FAN\_OUT = 5, 7, 9, and 11` all ran cleanly at 1 warehouse/2 terminals but never completed (150s+ wall time for a 20s run, under 15s of actual CPU work across all threads — a genuine stall, not slow-but-live progress) at 2 warehouses/4 terminals. `FAN\_OUT=27` (KiB1) was the smallest value confirmed to hold up at that load and at 4 warehouses/8 terminals; nothing here was tested past 8 terminals, so treat it as “more contention-sensitive than KiB2/KiB4”, not as unconditionally safe at arbitrary concurrency.

13.3 A benchmark methodology bug, found and corrected mid-investigation

An early benchmark (8 warehouses, 8s/point, 20 points across 5 sizes $\times$ 2 thread counts $\times$ gc on/off, on a 2 $\times$ AMD EPYC 7742 server) found scan p50 latency essentially flat across all five then-existing size variants (Tiny/Small/Medium/Large/Huge — the naming predating today’s KiB\emph{N} scheme) and concluded “Medium ties for best-or-tied-for-best on every metric measured.” That flatness finding was **wrong, as an artifact of the benchmark’s own duration**, not because the underlying effect does not exist. The engine’s always-on OLAP thread ages its scan snapshot by a configurable delay before scanning; the original 8-second points never let that delay reach more than 3–4 seconds, so the garbage-accumulation effect the doc comment above describes never actually got exercised within the timed window.

Re-run with a 30-second window and the **sweep** OLAP mode (isolating exactly the warehouse+district scan), `olap\_param=5` (delays 0,1,2,3,4,5s):

Delay	KiB32 (default)	KiB512
0 s	155 μ s	159 μ s
1 s	56 μ s	249 μ s
2 s	39 μ s	294 μ s
3 s	58 μ s	343 μ s
4 s	61 μ s	354 μ s
5 s	60 μ s	269 μ s

Both scans return the identical 88 rows every time — the latency gap is pure physical-record-touched overhead, not output size. KiB512 is consistently 4–6 $\times$ slower than KiB32 past the first (near-zero-garbage) point, confirming the original “ $\sim 100\% \rightarrow \sim 21\%$ of baseline OLAP throughput” claim in the source comment and correcting the earlier “flat” finding: it was an artifact of too-short aging, not an absent effect. The lesson generalizes past this one benchmark: *a benchmark whose measured window is shorter than the mechanism’s own time constant will report “no effect” regardless of whether one exists* — exactly the caution the catalog documents already give about not trusting a short/noisy sample, applied here to duration rather than run count.

A system-wide diagnostic added during the same investigation (`mv\_test::SCAN\_TRACE`, off by default) found **5.74 $\times$ records-visited per record-matched**, identical at both leaf sizes tested — expected, since the other twelve standard-sized tables’ read volume swamps warehouse-/district’s share of the global counter, but a real, general finding worth keeping in mind: this engine’s reads, at this write-heavy TPC-C scale, touch 5–6 $\times$ more physical records than they return, on average, across the whole workload, not just this one optimization’s target tables.

13.4 Recommendation

KiB32 remains the correct general-purpose default: it is the current `\#[default]` variant, and per the doc-commented measurements above it captures most of the achievable contention win while keeping OLAP scan cost at parity with the untouched baseline. Reaching for KiB64/KiB512 is a deliberate trade of scan latency for root-contention headroom that should be a conscious choice for a specific deployment’s read/write mix, not a default.

14 Rejected, reverted, or deliberately optional experiments

An accurate account of this implementation has to include what did not survive. Some of the entries below are already referenced above; this section collects all of them, plus two GC-related attempts that are not documented in any other file in this repository and exist only in this investigation’s own history.

Location	Attempt	Why attempted	Observed result	Failure mode
d565b2d–5c26abb	ORWC optimistic read/write coupling	Non-blocking upgradeable writers alongside free readers.	~460 lines ultimately removed, no retained gain.	Repeated upgrade and dangling read it duplicate OLC sound distinct guard GC did not know the guard’s lifetime could invalidate i
45cb048	Hold one long-lived range-scan block guard	Avoid re-borrowing blocks during iteration.	Not retained.	Equal-boundary c cannot prove histo intervals superseded stress returned d keys.
§11 rejected set	Eager child-list reuse, small-page dedup, hybrid traversal, lean frames	Remove allocation/hash work, release internal pages early.	An experimental version measured 18–20% faster.	Tight retry loop backoff, starved v under contention.
0659d34	Retry an alternate merge candidate immediately	Increase merge success on first-candidate failure.	No gain retained.	More SMOs ampli contention while off-by-one bug re unaddressed.
Companion to 85ac1b0	Lower the leaf overflow trigger for more compaction headroom	Reduce chance of a full leaf meeting a pending write.	Stress hung.	Since an open tra may read any key in case this reader is indistinguishab worst case from p nearly everything self-reinforcing co livelock, just com more expensively global-watermark replaced.
tx\ _context. rs/smo.rs, per-reader LCB check	Replace the global oldest-snapshot watermark with a per-reader liveness check (<code>any_live_reader_missing</code>)	Avoid coupling every leaf’s reclaim to the single slowest live transaction anywhere.	Instrumented run: 517,748 blocked deletes in 12 seconds, one fixed reader never advancing, no progress.	An early, ratio-ba compaction trigger reachable at a mu bar than the exist 100%-full trigger, regime (a same-tr write to a leaf-ro replaced mid- tra by a different key compaction) the surrounding abort/write-set m was never proven Every real caller routes through LocalBatch’s per buffer, even at ba
smo.rs, unsafe\ _degree\ _root	Trip Overflow early once a leaf-root’s dead-record count alone reaches 50% of capacity (not only at 100% physical fullness)	Reduce OLAP scan cost on hot tiny tables (see §13) without waiting for full-capacity compaction.	Scan latency win confirmed the mechanism (KiB512: $\sim 250\text{--}350\mu\text{s} \rightarrow \sim 145\text{--}195\mu\text{s}$), but cargo test --release run repeatedly surfaced an intermittent ($\sim 30\text{--}40\%$ of runs) stocks_ytd invariant failure, single-threaded, reproduced in isolation.	
Thread-local raw lock-free WAL buffer	Reuse encoding memory on the caller thread	Avoid one allocation per write.	No production gain.	

Location	Attempt	Why attempted	Observed result	Failure mode
6486d97, mimalloc feature	Cheaper cross-thread frees for the sharded WAL	YCSB-A +2.3–3.0%; TPC-C fell 3.3–4.9% across every backend.	Workload-dependent allocator trade-off.	Not a universal w
Very large BigTreeSize leaves	Defer compaction on the hottest tiny tables aggressively	Short tpmC results were flat and restart counts kept improving with size.	Aged scans at KiB512 were 4–6× slower than KiB32.	Diminishing/inve returns past the r sweet spot.

Table 2: Rejected, reverted, or deliberately optional experiments.

15 Open issues, as of this writing

These are real, unresolved gaps surfaced by this investigation’s own history. They are reported here, distinctly from the reverted-experiments table above, because each one is a currently-*unfixed defect* rather than a tried-and-abandoned idea — an honest account of the implementation has to include both categories separately.

- **A pre-existing `split()` livelock, independent of the two reverted GC attempts above.** While batch-verifying an unrelated fix, `concurrent_multi_step_transactions_across_a_small_shared_key_set_never_lose_an_update` hung for 500+ seconds (expected $\sim 3\text{--}4\text{s}$) with steadily growing RSS at only $\sim 3\%$ CPU — reproduced on *completely unmodified* HEAD at roughly 1/25 runs ($\sim 4\%$). Some data/version distribution can make `split()`’s `ByVersion` retry loop never converge, matching that function’s own doc-commented predicted failure mode (“repeats forever”). A real fix needs either (a) a fallback to a genuine physical `ByKey` split when `ByVersion` compaction fails to shrink the survivor set enough, or (b) per-transaction read-set tracking so protection scopes to keys a reader actually touched rather than everything after its `ts_start`. Both are materially larger than a single-predicate change. **Open**
- **A second, independent GC-visibility gap in `record_survives_gc`.** OSIC visibility is LCB-based, not a raw `ts_start` comparison: a reader’s own `ts_start` being numerically later than a foreign write’s does not guarantee that write is visible to the reader yet — `is_snapshot_live` can go false (the writer committed and unregistered) while some *other*, still-registered reader’s snapshot still needs to resolve to that exact superseded version. Both attempted fixes for this (the global-watermark version and the per-reader LCB version, described in §14) were reverted because they reproduce a self-reinforcing compaction livelock. The gap itself remains — documented in-code as a comment, with no behavior change — and is the confirmed root cause of a residual $\sim 7\text{--}15\%$ -under-load flake in `verify_concurrent_shared_keys` (diagnosed with hard instrumentation evidence: the only surviving record for a missed key was a fresh insert not yet visible to the reader, with the previously-visible version already gone, immediately after a burst of `ByVersion` compactions on that leaf). **Open**
- **TPC-C’s OCC conflict rate is very high** at scale (Payment $\sim 71\text{--}76\%$ of attempts conflict and are discarded; New-Order $\sim 47\%$, at 8 warehouses/64 terminals) — pure wasted traversal/split work, unrelated to and unaddressed by the WAL work in §10. **Open**
- **Contention split and XMerge** (Alhomssi & Leis’s B-tree contention/space-management techniques, from the same paper OSIC comes from) remain unported. Restart-attribution instrumentation run at this project’s default TPC-C scale found root-level contention

dominates (before the `BigTreeSize` fix); after that fix, the remaining restarts are mostly internal/routing-page (`checked_live_version`) failures on a small number of high-fan-in pages with hundreds of distinct keys funneling through each — exactly the “many distinct keys sharing one page’s latch” pattern contention split targets. No per-page restart/contention counters exist yet to drive it. **Open**

- **The per-write WAL allocation on `WalWriter`’s cross-thread path** is smaller after the framing-hint fix (§10) but not eliminated: a fresh buffer is still allocated per write and freed on a different thread. A bounded per-shard buffer pool, with buffers returned right after `flush_loop` copies their bytes (well before the `pwrite/fsync`, so the pool need not wait on durability), is the identified but unimplemented fix. **Open**

16 Fresh measurements, this session

Everything in this section was run on the machine this document was written on (AMD Ryzen 7 2700X, 8 cores/16 threads, rustc/cargo 1.97.1) against the exact repository state at commit 64e4380 (2026-08-12), specifically to cross-check the historical claims above against the code as it exists today, not to replace them — the historical numbers above were mostly measured on different, often larger/newer, hardware, and are kept as-is with their original attribution.

16.1 Mechanism presence check

Every mechanism this document describes as currently active was confirmed present in the source at 64e4380 by direct inspection/grep, not assumed from the documents alone: the self-overwrite fast path (`insertion_stamp()==stamp` in `insert\_on\_tree/update\_on\_tree`), `BigTreeSize/TreeClass` (now an eight-variant enum, richer than any prior document describes — §13 reflects the current, not the originally-documented, version), `RESTART\_TRACE` (`false` by default, with its OOM-fix reset call intact), `EXP\_SPIN\_CAP/JITTER\_BACKOFF\_THRESHOLD` in the `SmartCell` backoff schedule, `LockFreeWalWriter`, and the point-read fix (no `skip\_while` shortcut remains in `key\_point\_read\_from\_root`, now committed as c0fe172 — it was still uncommitted as of the most recent prior investigation notes available). The two GC-livelock revert attempts documented in §14 were confirmed genuinely absent from the current source (`any\_live\_reader\_missing/is\_visible\_nocache` do not appear anywhere in `src/`), consistent with those reverts having held.

16.2 Full test suite

`cargo test --release` at 64e4380: **151 passed, 0 failed, 0 ignored, finished in 5.71s** (wall clock for the whole invocation including the two empty loom-test binaries: 6.14s). This is a healthy, fast-feedback current state, consistent with the “smoke-scale by default, `CMVBT_FULL_BENCH=1` for full scale” design described in §12. It does not, by design, exercise either open livelock in §15 deterministically — both are reported as load-sensitive/rare, and this single run neither reproduces nor disproves them.

16.3 Leaf layout benchmark, independently reproduced

Following the exact reproduction recipe in §3 (`rustc --edition=2024 -C opt-level=3 -C target-cpu=native tools/leaf_layout_bench.rs, taskset -c 0`), five full runs of the binary were taken (the benchmark internally already samples for 750 ms per operation; five *process* runs were taken to match the original report’s “median of five” protocol at the process level). The five runs agreed with each other to well under 1% — this machine was otherwise idle, so the tight agreement reflects a quiet machine, not that the underlying effect is small.

Operation (62.5 MiB multi-leaf sweep)	AoS	Split	Speedup
Point: latest visible	10.90 Mops/s	11.27 Mops/s	1.033×
Point: 1-version fallback	7.86 Mops/s	8.43 Mops/s	1.073×
Point: 3-version fallback	4.66 Mops/s	4.32 Mops/s	0.927×
Scan: full leaves	245.2 Mslots/s	236.2 Mslots/s	0.963×
Scan: 50% ranges	244.5 Mslots/s	237.3 Mslots/s	0.971×
Scan: 20% ranges	254.0 Mslots/s	236.0 Mslots/s	0.929×

This both confirms and diverges from the original report. The direction of the point-lookup effect replicates (split is faster for the common “latest visible” and “one fallback” cases), but at roughly a third of the claimed magnitude, and the 3-fallback case and two of three scan variants *reverse sign* entirely versus the original Ryzen 9 5900X report. The most plausible explanation is genuinely the hardware, not a methodology error: this machine’s CPU (Ryzen 7 2700X, Zen+, 2018) has a smaller, higher-latency L3 cache and weaker prefetchers than the original report’s Ryzen 9 5900X (Zen 3, 2020) — the split layout’s win is specifically a cache-pressure/locality effect (fewer unrelated cache lines touched per rejected key), so a CPU with less cache to relieve pressure on, or a memory subsystem that already dominates the access cost regardless of layout, would show a smaller and noisier version of the same effect, plausibly crossing zero for the scan case where the layout-independent cost (touching every byte) already dominates. This is reported in full, with the divergence unresolved rather than explained away, because a microbenchmark result that fails to reproduce cleanly on different hardware is itself exactly the kind of finding §3’s own “Reading the performance claims” caution warns a reader to expect and account for.

16.4 WAL writer throughput, independently reproduced

`cargo test --release of compare\_wal\_writer\_vs\_lockfree\_throughput` (the same test the original synthetic comparison in §10 used, but here run in a full release+LTO build rather than debug, with a fuller thread sweep {1, 2, 4, 8, 16} and batch-size sweep {1, 4, 16, 64, 256}, 20,000 ops/thread):

Backend	@1 thread	@8 threads	@16 threads
WalWriter (4-way sharded channel)	700,592 ops/s	2,607,583 ops/s	4,339,121 ops/s
LockFreeWalWriter, unbatched	384,583 ops/s	839,536 ops/s	763,438 ops/s
LockFreeWalWriter + LocalBatch(4)	1,021,650 ops/s	2,371,383 ops/s	2,459,184 ops/s
LockFreeWalWriter + LocalBatch(16)	1,509,518 ops/s	4,898,065 ops/s	5,563,703 ops/s
LockFreeWalWriter + LocalBatch(64)	1,475,161 ops/s	5,979,338 ops/s	7,578,144 ops/s
LockFreeWalWriter + LocalBatch(256)	1,515,828 ops/s	6,386,070 ops/s	8,678,745 ops/s

This reproduces the qualitative story in §10 cleanly: unbatched lock-free writes lose to the sharded channel past a handful of threads, and per-thread batching is what actually wins, monotonically improving with batch size at 16 threads. The absolute numbers are considerably higher than the original debug-build report (e.g. sharded-channel at 16 threads: 4.34 M ops/s here vs. the original 1.35 M ops/s; batch-64 lock-free: 7.58 M ops/s here vs. 5.09 M ops/s originally) — expected and not a discrepancy worth worrying about, since this run used a full `opt-level=3/lto=fat/codegen-units=1` release build (this crate’s actual `[profile.release]`) against the original’s stated debug build. The *ranking* between backends is what the original report’s conclusion rested on, and that ranking reproduced exactly, magnitudes aside.

17 Combined measured gains at a glance

This table repeats only controlled, repository- or session-backed quantitative results, extended from the earlier catalog’s own closing table with this session’s fresh reproductions. It is the correct table to cite when a numerical gain is required; do not attribute a combined-batch row to one component in isolation.

Optimization set			Workload			Measured change	Qualification
4 KiB (b29d347)	base-block	tuning	Allocation / TLB profile			Alignment 25%→100%; dTLB load misses −9 to −12%	Exact object l pendent.
CRC32 + effective native build + 4-way WAL sharding			TPC-C, 16/64 threads			+7.4% / +17.6%	Combined resu assign all gai component.
Same three WAL/build changes			YCSB-A, 16/64			+25.4% / +68.7%	Write-heavy workload.
Same three WAL/build changes			YCSB-E, 16/64			+16.2% / +49.8%	Scan-heavy wi component.
Same three WAL/build changes			YCSB-F, 16/64			+20.7% / +70.1%	Read-modify-v
Ticket-less WAL + exact buffer hints			TPC-C / YCSB-A			+12.6% / +25.3%	Same short ing setup, b together.
Ticket-less WAL + exact buffer hints			Synthetic WalWriter, 16 threads			+79%	8-byte payload the removed t cation unusua nant.
Lock-free local batch-64 (original, debug build)			Synthetic	WAL,	16	5.09M vs. 1.35M ops/s	Unbatched loc slower; see this release reprodu low for the sa ing at different magnitudes.
This session: lock-free local batch-256 vs. sharded channel, re- lease+LTO			Synthetic	WAL,	16	8.68M vs. 4.34M ops/s (un- batched lock-free: 0.76M)	Same ranking original de result; magnit directly co across build pr 40 alternating after warmup.
Zero-copy streaming range termi- nal			200k-row full scan			1.165–1.173×; ~+16.8%	Indicative sh tended runs; ranges only.
Range-before-visibility predicate			CH Q4 / Q5			~+18% / ~+11–13%	Absolute put, implementation only.
Specialized count terminal			Full scan			~49–53M rows/s	OLAP scan th at parity with at this size.
BigTreeSize KiB32 default vs. untouched leaf			District/warehouse root restarts			~4.4×/~2.6× reduction	

Optimization set	Workload	Measured change	Qualification
BigTreeSize KiB512 vs. KiB32	Aged OLAP scan latency (30s window, sweep mode)	4–6× slower past 1s of aging	Corrects an e window benchr read as “flat.”
Full-row YCSB-A batch	YCSB-A	2.52M→3.11M ops/s (+23.5%)	200k rows, 4 3s.
Standard single-field + alias Zipf + hit-only path	YCSB-A (standard) / YCSB-C	4.81M ops/s / 7.16M→14.82M ops/s (+106.8%)	Different s than full-row r directly comp it.
Scan-loop and measurement batch	YCSB-E	4.59M→6.22M ops/s (+35.4%)	Combined bat latency samp scan in 1,024.
Mimalloc feature	YCSB-A / TPC-C	+2.3–3.0% / –3.3 to –4.9%	Kept optional universal optim
This session: leaf layout, split vs. AoS, Ryzen 7 2700X	Point lookup / scan, 62.5 MiB multi-leaf	1.03–1.07× / 0.93–0.97×	Smaller and p versed vs. t nal Ryzen 9 5 port — see the hardware-s discussion.
This session: full test suite, commit 64e4380	Development test run	151 passed, 0 failed, 5.71s	Correctness/op result, current document’s wr

18 Timeline

Table 4 indexes the commits discussed in this document chronologically, for cross-reference against `git log`. It is provided as a navigational aid only — the organizing structure of this document is architectural layer and dependency, per §1, not this chronology.

Commit	Date	Theme
acdc7a8	2024-02-14	GC started (§7)
95e149f	2024-02-29	GC active-tx index moved to a concurrent B+-tree (§7)
d565b2d–	2024-01–	ORWC implemented, iterated, deprecated, purged (§4)
5c26abb	2024-12	
48c4ded,	2024-05–	Fence removal / ordering tightening (§4)
a7bb29c,	2025-04	
a518eed		
bea1117	2025-06-05	GC-driven update-in-place (unrelated mechanism, §6)
17b8f5f–	2025-12–	Pre-OSIC commit/visibility, thread-watermark patch-
836a1db	2026-01	ing (§10)
34555e2	2026-06-20	Overflow-trigger tuning (§5)
2a2be93,	2026-06	Further ordering relaxation (§4)
40b00fd		
714a82f	2026-07-10	OSIC/WAL adopted wholesale (§10)
3e8b837,	2026-07-	Group commit, ELR removal, log truncation (§10)
ab10660,	13/15	
73cc4a5		

Commit	Date	Theme
5f92e9f	2026-07-15	Tables-as-shards (§9)
7f2bd09	2026-05-21	Cache-line isolation for node tag (§4)
45c7c21/fd92f38	2026-07-21	Conservative child latch removed; retired-bit race fix (§4, §7)
d0e0977		
f9fc960	2026-07-23	GC own-shard-first + steal (§9)
ecc9900	2026-07-27	Racey-stamp + dead-page-key collision fixes (§7)
fe03d90	2026-07-28	Obsolete-mark removed; overlap-based liveness (§7)
4bc78de	2026-07-28	Merge-overflow accounting fix (§5)
7432525–fc8806b	2026-07-31	TPC-C crash hardening week (§5)
160186e, 85ac1b0, fbedd74	2026-08-01	Repeated-key abort fix; self-overwrite fast path ; hot-key repro (§6)
2a2607e, dc30111, dcb5abe, 0659d34	2026-08-02	Split off-by-one and merge/boundary-search fixes (§5)
eac6e40, b29d347	2026-08-02	Cache-padded worker atomics; FAN_OUT page alignment (§9, §5)
4a1a50a	2026-08-09	WAL channel sharding, table-driven CRC32 (§10)
ebdd789, 9ced1a3, efc3773	2026-08-10	Lock-free WAL backend; RESTART_TRACE OOM; cross-shard race; ticket/framing fixes (§10)
8e48ae5	2026-08-11	Range-before-visibility reorder; dyn removal (§11)
c0fe172	2026-08-11/12	Point/range-read miss fix: physical order does not track ts_start order (§11)
1f6d666, c525eba	2026-08-09/12	BigTreeSize introduced, then extended with sub-4KiB variants (§13)

Table 4: Chronological index of commits discussed in this document.

19 Conclusion

Read end to end, the history of this codebase tells one consistent story at every layer: an initial design gets something working with coarse-grained locks, mutexes, or bolted-on atomics/fences; profiling or ThreadSanitizer then reveals exactly where that coarseness costs correctness (a race) or throughput (contention); and the fix is, more often than not, not “add a lock” but “restructure the data so the hazard cannot occur” — append-only pages instead of in-place edits, interval-overlap-derived liveness instead of a mutable obsolete bit, per-worker sharded bookkeeping instead of one shared structure, instant in-memory commit instead of commit-time fsync. The four foundational optimizations named in §2 are the load-bearing instances of that pattern; the self-overwrite trick in §6 is a small, local instance of exactly the same shape one layer up: rather than adding machinery to garbage-collect or specially handle a hot key’s pile of uncommitted versions, the fix removes the pile-up from occurring in the first place.

An equally important, less flattering thread runs alongside it, and this document has tried not to understate it: not every attempted instance of this pattern succeeded. Two separate attempts to close the GC-visibility gap in §14 each independently rediscovered the same self-reinforcing compaction livelock from a different angle, and both were correctly reverted rather

than shipped with a rare, hard-to-attribute correctness cost. A benchmark’s own measured window can itself be wrong in a way that produces a confident, false “no effect” conclusion (§13), and a microbenchmark’s headline percentage can fail to reproduce, in both magnitude and sign, on different hardware without either measurement being in error (§16). A codebase’s real state, at any given moment, is the sum of the optimizations that shipped, the ones that were tried and correctly reverted, and the ones that are known to still be broken — this document has tried to give an accurate account of all three, not just the first.

References

- [1] Becker, B., Gschwind, S., Ohler, T., Seeger, B., Widmayer, P. *An asymptotically optimal multiversion B-tree*. The VLDB Journal, 5(4):264–275, 1996.
- [2] Tonta, A., Seeger, B., Soisalon-Soininen, E. *Multiversion Concurrency Control for Multiversion B-Trees*. arXiv preprint arXiv:2606.09133, 2026.
- [3] Alhomssi, A., Leis, V. *Scalable and Robust Snapshot Isolation for High-Performance Storage Engines*. Proceedings of the VLDB Endowment, 16(6):1426–1438, 2023. doi:10.14778/3583140.3583157.